

Youtube: <https://www.youtube.com/channel/UCKxWlkyWoxfa3EOVXzMhDPQ>

Mobile Pen testing with ADB

ADB (Android Debug Bridge): it's a powerful **command-line tool** that allows you to **communicate with an Android device** (either physical or emulator) from your computer.

Android Virtual Device (AVD):

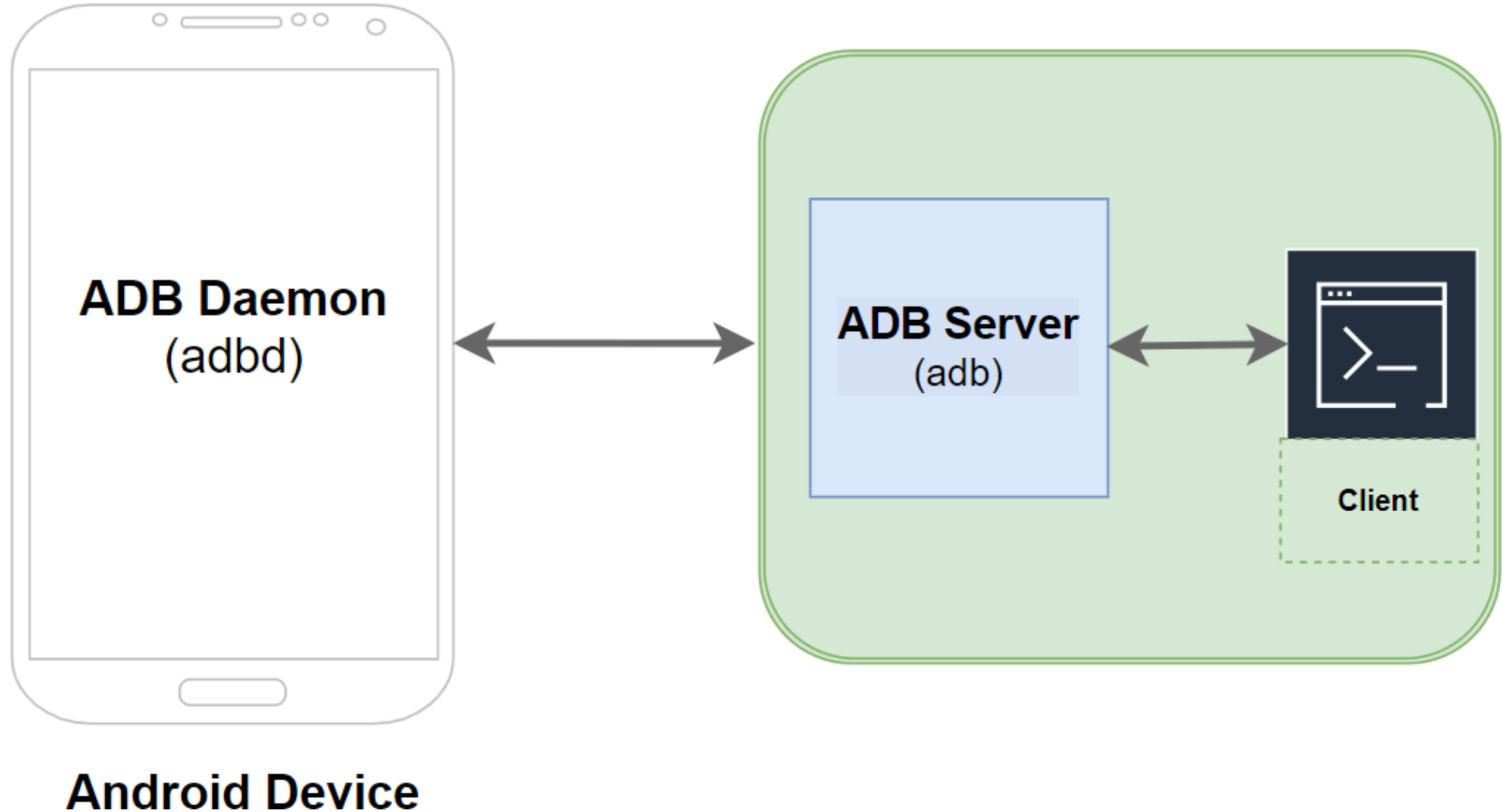
A configuration for the Android Emulator that simulates different physical Android devices, allowing developers to test their apps in a variety of settings. You can create and manage AVDs using the Device Manager in Android Studio.

Purpose

ADB is mainly used by developers, security researchers, and testers to:

- Debug Android apps
- Access device logs
- Install/uninstall APKs
- Run shell commands
- Transfer files between PC and Android
- Control the device remotely

Android Debug Bridge



How It Works

ADB works as a **bridge** between your **computer** and the **Android device** through **USB** or **Wi-Fi** connection.

It has **three main components**:

- 1.Client** – Runs on your computer (you type ADB commands in terminal or command prompt).
- 2.Daemon (adbd)** – Runs on the Android device and listens for commands.
- 3.Server** – Manages communication between client and daemon.

When You'd Use ADB

- When you're **developing or testing Android apps**.
- To **access hidden system files** or **perform forensics/reverse engineering**.
- For **rooting, flashing, or custom ROM installation** (advanced use).
- To **debug or analyze malware APKs**.

Connect your device with adb



Step 1: Install ADB on Your Computer

◆ For Windows

So download Android Studio

<https://developer.android.com/studio#get-android-studio>



Step 2: Enable Developer Options on Your Android Device

1. Open **Settings** → **About Phone**
2. Tap **Build Number** 7 times until it says *"You are now a developer!"*
3. Go back to **Settings** → **Developer Options**
4. Turn on **USB Debugging**



This allows your PC to communicate with your Android device.

adb location in windows

C:\Users\ksham\AppData\Local\Android\Sdk\platform-tools



Step 4: Verify the Connection

adb devices

```
C:\Users\ksham\AppData\Local\Android\Sdk\platform-tools>adb devices
List of devices attached
1365eb74          device
```

```
adb shell
ls /sdcard/
```

```
C:\Users\ksham\AppData\Local\Android\Sdk\platform-tools>adb shell
RE606FL1:/ $ ls /sdcard/
A-personal\ data
AWS\ Certified\ Solutions\ Architect\ -\ Associate\ certificate.pdf
Alarms
Android
Audiobooks
Browser
ColorOS
CompTIA\ Security+\ ce\ certificate.pdf
DCIM
Documents
Download
ECC-CEHPRACTICAL-Certificate.pdf
Fallout.S01
Fonts
RE606FL1:/ $
GLA\ Documents
Instagram
MX\ File\ Transfer
MY\ photos
Mohsin_Qureshi_resuME\ -1.pdf
Movies
Music
MxPlayerAd
My\ Dockuments
My\ Documents
My\ pic
Notifications
Phone
Pictures
Podcasts
Recordings
Ringtones
Saved\ Searches
Subtitles
Supacell
Telegram
TeraBoxDownloads
blockcanary
oua_classifier
port-forward.txt
whatsa
```

Close Emulator: adb -s emulator-5554 emu kill

Connect with IP Address (ADB over WIFI)

1) Prep (USB connected)

Make sure:

- Phone is connected by USB.
- USB debugging is enabled.
- **adb devices** shows your device.

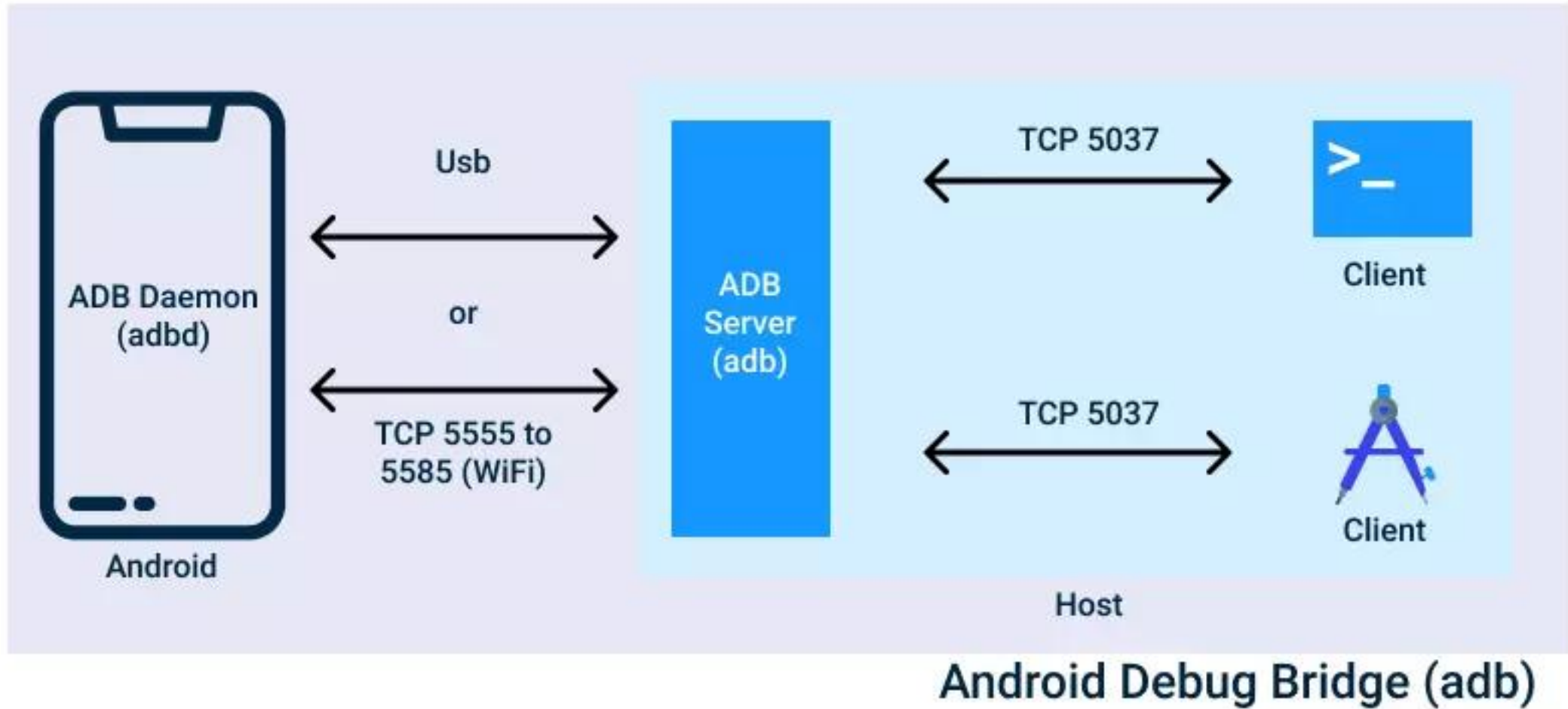
2) Tell the device to listen on port 5555

With USB connected, run:

```
adb tcpip 5555
```

This restarts the ADB daemon on the device and switches it to listen on TCP port **5555**.

How does it work?



3) Find the phone's IP address on the Wi-Fi network

Options (pick one that's easiest):

- **On the phone:** Settings → About phone / Status → Wi-Fi IP address.
- **Via ADB:** (works from terminal)

```
adb shell ip route
```

4) Connect ADB over Wi-Fi

```
adb connect 192.168.2.40:5555
```

Check: adb devices

Now the device should appear as 192.168.2.40:5555 device.

You can now **unplug the USB cable** — ADB stays connected over Wi-Fi.

5) Revert back to USB-only (secure the device)

```
adb disconnect <PHONE_IP>:5555  
adb -s <PHONE_IP>:5555 usb
```

The adb ... usb command tells the device to switch back to USB mode.

ROOT Android Virtual Devices: Your 1st Step to Mobile Pen-Testing

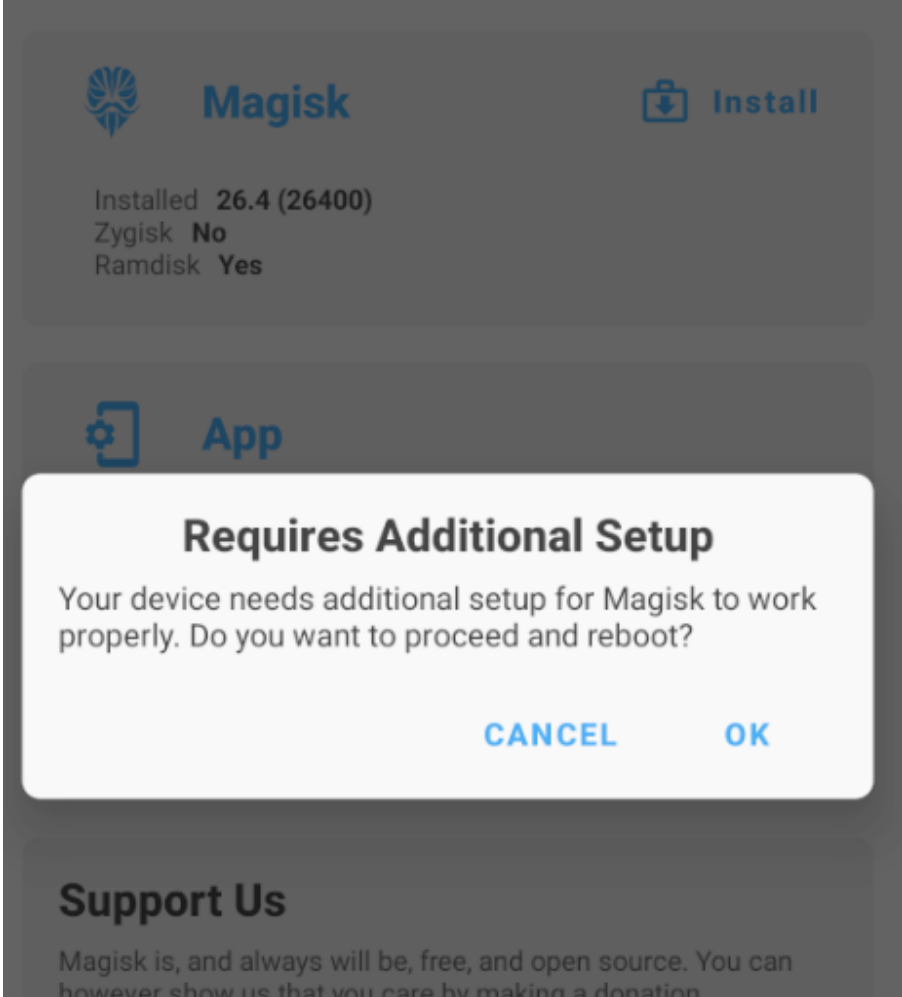
Root AVD

```
PowerShell 7.5.4
PS C:\Users\ksham\Music> winget install --id Git.Git -e --source winget
Found an existing package already installed. Trying to upgrade the installed package...
No available upgrade found.
No newer package versions are available from the configured sources.
PS C:\Users\ksham\Music> |
```

```
git clone https://gitlab.com/newbit/rootAVD.git
cd rootAVD
.\rootAVD.bat listAllAvds
```

```
./rootAVD.bat system-images\android-34\google_api_playstore\x86_64\ramdisk.img
```

After reopen AVD open Magisk apk file



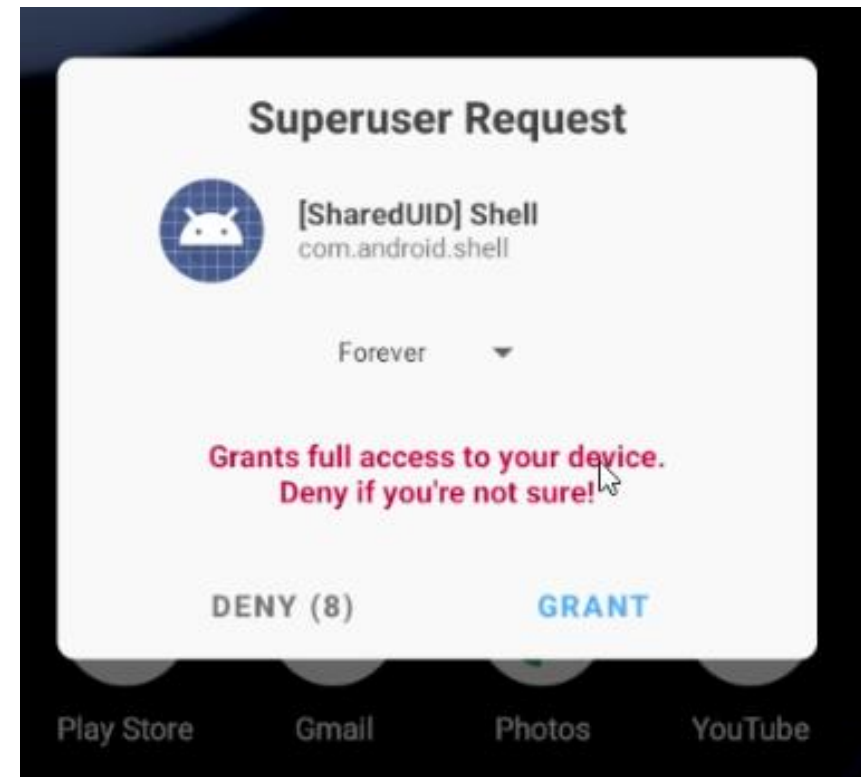
Now click on **ok** and wait for restart

After restart

adb devices

adb shell

su (when you run su command click on **GRANT**)



What Is Rooting?

Rooting gives you **administrator (superuser)** access to your Android system.

With root, you can:

- Modify system files
- Uninstall preinstalled (bloatware) apps
- Run advanced tools (e.g., firewalls, ad-blockers, custom ROM features)
- Do penetration testing or forensics (like in ethical hacking setups)



⚙️ SuperSU – The Classic Root Manager

📌 Overview:

- **Developed by:** Chainfire
- **Type:** *System-root method*
- **Released:** Around 2012 (popular until Android 6–7)



How It Works:

- SuperSU **modifies system partitions** to gain root access.
- It installs a su binary directly in /system/bin or /system/xbin.
- Root access is managed via the SuperSU app.

Advantages:

- Stable and simple to use
- Works well on older Android versions
- Provides good control over which apps get root access

Disadvantages:

- **System modifications:** Changes /system → causes issues with OTA updates
- **SafetyNet fails:** Google SafetyNet detects system modifications → apps like Google Pay, Netflix, banking apps won't work
- **No longer updated:** Chainfire discontinued it → outdated for Android 8+

⚙️ Magisk – The Modern Root Solution

📌 Overview:

- **Developed by:** topjohnwu
- **Type:** *Systemless root method*
- **Released:** 2016 onwards (actively maintained)



How It Works:

- Magisk roots your device **without altering /system** — all changes happen in the boot image.
- It uses **MagiskSU** (similar to SuperSU) but runs in a “systemless” way.
- Includes **Magisk Manager / Magisk App** to manage modules and root permissions.

Advantages:

- **Systemless root:** Keeps /system untouched → safer and reversible
- **Passes SafetyNet:** Apps like banking or Google Pay can still work
- **Magisk Modules:** You can install tweaks like:
 - Ad-blocking
 - BusyBox
 - System UI mods
 - Audio mods (Viper4Android)
- **Active development** → supports Android 14+
- Easy to **unroot or hide root** from apps

Disadvantages:

- Slightly complex installation (especially on newer phones with A/B partitions)
- Can conflict with some OTA updates (if not handled correctly)

Master adb Commands

```
adb devices
```

```
adb install Divaapplication.apk
```

```
adb shell pm list packages | findstr diva
```

```
adb uninstall jakhar.aseem.diva
```

```
adb exec-out screencap -p > screenshot.png
```

```
adb shell screenrecord /sdcard/demo.mp4
```

```
adb pull /sdcard/demo.mp4
```

```
adb push myfile.txt /sdcard/
```

```
adb reboot
```